16. Web Crawler

> **Interviewer:** Design a web crawler. Start from a handful of seed URLs — say `wikipedia.org`, `nytimes.com` — and crawl outward: fetch a page, extract its links, fetch those too, forever. The goal is a store of page content that something like a search index can read later. Go.

Candidate: Fun one — this is less "scale the reads" and more "don't destroy the internet while politely eating all of it." Let me clarify before I size anything.

---

1. Clarifying the requirements

> **Candidate:** A few questions up front: 1. Target scale — millions of pages, or billions? 2. Do we re-crawl pages to catch updates, or is one pass enough? 3. Is there a hard rule around never hammering one site — `robots.txt`, per-domain limits? 4. Any real-time freshness requirement, or is "eventually fresh" fine? 5. Who's downstream — does a search indexer call us, or do we just drop content somewhere for it to read later?
>
> **Interviewer:** Billions of pages, continuously — 10 billion, re-crawled every couple of weeks. `robots.txt` is non-negotiable, and you must never overload a single site even if it means crawling that site slowly. No real-time requirement. Downstream is a search indexer that reads whatever you store — it doesn't call you, you don't call it.

Candidate: Good. Requirements:

Functional - Start from seed URLs, download each page's HTML. - Extract links, feed newly-discovered URLs back into the crawl. - Store content somewhere a downstream consumer can read by URL. - **Never re-crawl a still-fresh page** — dedup. - **Respect `robots.txt`** and per-domain politeness.

Non-functional — this is where it's interesting - **Massive scale**: billions of pages, trillions of discovered links over time. - **Politeness over speed**: a fast crawler that gets our IPs banned has failed, even though it was "fast." This dominates every other concern. - **No single point of control**: thousands of machines running for weeks must make forward progress while individual machines and sites keep failing. - **No user-facing latency**: nobody is waiting on a response — a background pipeline, not a request/response API. This changes almost every choice downstream. - **Bounded blast radius from crawler traps**: some sites generate infinite URLs; one bad site must never consume the whole crawl budget.

---

2. Estimation — sizing the frontier and the fleet

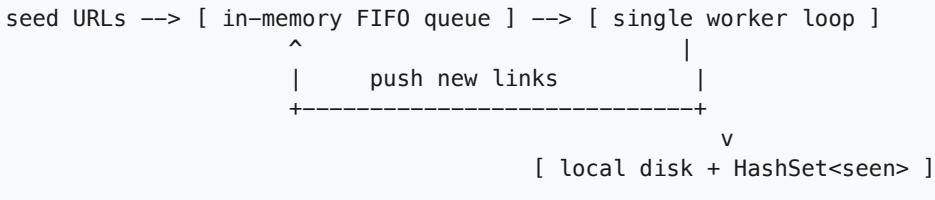
```
Target: 10 billion pages, full re-crawl cadence ~= 14 days.

Throughput: 10e9 / (14 * 86,400s) ~= 8,300 pages/sec average
            design for ~3x peak -> ~25,000 pages/sec sustained at busy times

Bandwidth (avg page ~100 KB kept): 25,000 * 100 KB ~= 2.5 GB/sec peak

Storage: 10B * 100 KB ~= 1 PB raw (compresses to a few hundred TB)

URL frontier (discovered-but-not-fetched):
  each page yields ~10-50 links -> 100B-500B edges discovered over time.
  The LIVE frontier alone can hold billions of URLs at once, ~100 bytes
  metadata each -> hundreds of GB. Doesn't fit one process; churns constantly.

Worker fleet: ~50 pages/sec/worker (network+parse bound, not CPU)
  25,000 / 50 ~= ~500 worker machines at peak.

=> Two numbers define this design: (1) the frontier is too big/hot for one
   queue, (2) 500 workers pulling from it must never let more than ~1
   request/N-seconds land on any single domain, no matter how many workers exist.
```

Candidate: Unlike the burger giveaway or flash sale, the hard constraint isn't one hot key under a write storm — it's the **opposite shape**: enormous fan-out across millions of independent domains, each individually throttled, plus a shared "have I seen this" check that must run at frontier scale without becoming its own bottleneck.

---

3. A simple V1 (single machine, and why it dies)

> **Interviewer:** Sketch the dumbest version first.



Pop a URL, fetch it, save the HTML, extract links, push new ones back, mark seen locally.

Why it dies: - One machine can't hold billions of URLs in memory (the seen-set alone is tens of GB). - One machine can't sustain 25,000 pages/sec — we need ~500 workers. - **No cross-worker politeness.** 500 copies of this loop could all hit `nytimes.com` in the same second — exactly the failure mode we must avoid. - **No crash recovery** — kill the process and the whole frontier + seen-set vanish.

This points to three real problems to solve: a **distributed, durable frontier**, a **dedup mechanism that works at billions-of-entries scale**, and **per-domain politeness enforced centrally**, independent of fleet size.

---

4. Scaling out — the distributed crawler architecture

> **Interviewer:** Make it distributed. What's the real architecture?

```
                +-----------------------------+
                |   URL Frontier (sharded by   |
                |   DOMAIN, durable — Kafka     |
                |   partitions / distributed    |
                |   priority queue)              |
                +-----------------------------+
                   |shard A |shard B |shard C
                   v        v        v
             +---------+ +---------+ +---------+
             | Worker  | | Worker  | | Worker  |
             |  pool A | |  pool B | |  pool C |
             +---------+ +---------+ +---------+
(each worker, per fetch:)
1. robots.txt cache -- allowed?      --> no  --> drop
2. Bloom filter -- seen recently?    --> yes --> drop
3. per-domain token bucket -- token available? --> no --> requeue later
4. HTTP GET the page
5. store body -> object storage, keyed by hash(url)
6. update url metadata -> KV store (status=DONE, last_crawled=now)
7. extract links -> push new URLs back onto the Frontier
```

- **Frontier partitioned by domain, not randomly.** All of `nytimes.com`'s URLs land on the same shard, so one small set of workers (and one rate limiter) ever talks to that domain — the central decision, revisited in section 6.
- **Workers are stateless and horizontally scalable.** All real state (frontier, seen-set, crawl status, content) lives in shared durable stores, letting us go from 1 to 500 machines with no redesign.
- **No public API surface.** Nobody outside this pipeline calls it — the frontier queue and object store *are* the interface. Internal calls (e.g. "am I allowed to fetch this?" to a robots-cache service) are a good fit for internal gRPC, but that's a side detail.

---

5. The URL frontier — priority queues + politeness queues

> **Interviewer:** Design the frontier properly — how does it pick what to hand a worker next, out of billions of candidates?

Candidate: It answers two questions at once: **what's most worth crawling?** (priority) and **which domains are we currently allowed to hit?** (politeness). Two layers:

```
LAYER 1 — PRIORITY QUEUES (per tier, global)
  Q_high   : homepages, high-inlink-count, rarely-crawled
  Q_medium : normal pages, due for re-crawl
  Q_low    : deep pages, low importance, recently crawled
  (score on insert: importance (inlink count) + staleness (time
   since last crawl); higher score -> higher tier -> pulled sooner)
                          |
                          | a router pulls the next-highest URL
                          ▼
LAYER 2 — PER-DOMAIN POLITENESS QUEUES (one FIFO per host)
  queue[nytimes.com]   -> token bucket: 1 req / 2s
  queue[wikipedia.org] -> token bucket: 1 req / 1s
  queue[tiny-blog.com] -> token bucket: 1 req / 5s (default)

workers pull ONLY from domain-queues whose bucket currently has a token
```

- **Why two layers?** A single global priority queue would let a worker grab the globally top-priority URL even if it's the 50th `nytimes.com` URL this second — the overload we must avoid. Politeness has to be enforced **per domain**, independent of a URL's importance.
- **The politeness queue is the real bottleneck for popular domains — by design.** If `nytimes.com` has 50M pages, we accept it takes a long time, because the bucket caps us at, say, one request every 2 seconds, *no matter how many workers exist*. More workers help other domains, not this one.
- Mechanically: domain queues live in the same durable store as the frontier; the token bucket is a small counter per domain, refilled on a timer, checked before a worker gets a URL.

---

6. Dedup at scale — "have I already crawled this?"

> **Interviewer:** With billions of URLs flowing through, how do you check "already seen" without that check becoming its own bottleneck?

Candidate: Classic approximate-membership problem — a `HashSet<String>` or a SQL lookup doesn't survive this volume.

```
new URL found
     |
     ▼
BLOOM FILTER (in-memory, billions of bits, sharded like the frontier)
     |  "have I probably seen this URL in the last N days?"
     ├── "definitely NOT seen" ──► enqueue into frontier + set the bit
     └── "probably seen"       ──► drop (rare false positive: skips a
                                        page not actually crawled — fine,
                                        self-corrects next crawl pass)
```

Why a Bloom filter, not a naive hash set or DB lookup? - A **plain** `HashSet` of billions of URL strings needs tens-to-hundreds of GB of actual strings/hashes — doesn't fit one machine, doesn't shard cleanly across stateless workers. - A **DB uniqueness check** (`SELECT WHERE url = ?`) is correct but at ~500B discovered edges, that's 500B round trips to a store that should be serving real crawl-state traffic, not absorbing a membership check that doesn't need perfect accuracy. - A **Bloom filter** trades a small, tunable false-positive rate (e.g. 0.1%) for O(1), memory-only checks — no network hop, no disk I/O.

Sized for ~20B entries at 0.1% FP, it costs tens of GB of RAM, shardable by domain like the frontier. The failure mode (skipping an uncrawled page) is harmless — it's picked up on the next pass. - The durable `urls` KV table remains the source of truth for crawl **state**; the filter is just a fast pre-filter that avoids hitting that table for most "seen" answers.

Near-duplicate content (different URL, same page — mirrors, tracking params) is a separate check: hash the fetched *body* (e.g. content hash / SimHash) and compare against recent hashes — catches duplication the URL-keyed filter structurally can't.

---

7. Politeness in depth — `robots.txt`, rate limits, and caching

> **Interviewer:** Walk me through the mechanics before a single byte is fetched from a new domain — and what do you cache, since re-checking everything per page would be wasteful?

```
first time we see domain "example.com":
    GET https://example.com/robots.txt
          |
          ▼
    parse disallowed paths + optional Crawl-delay hint
          |
          ▼
    cache parsed rules (TTL ~24h — robots.txt rarely changes; a
    per-page re-fetch would DOUBLE our request count to every site
    on earth for zero benefit)

every subsequent URL on example.com, before fetch:
    1. disallowed by cached robots rules?  -> skip if so
    2. domain's token bucket has a token?  -> wait/requeue if not
    3. DNS: resolve host -> IP (cached locally, honoring the DNS TTL —
       re-resolving on every fetch at 25k/s is pure waste)
    4. fetch
```

- **The token bucket must be shared across every worker touching that domain**, not per-worker — otherwise 10 workers each self-limiting to "1/sec" still add up to 10/sec against the site. This is exactly why the frontier partitions by domain: it makes a shared rate limiter a local property instead of a distributed-coordination problem.
- **Both caches (robots, DNS) follow the same shape:** fetch once, reuse many times, expire on a timer, and treat a cold cache as harmless — a restarted worker just re-fetches once at small cost. Only the `urls` KV table and the Bloom filter must survive a restart; these caches are pure optimization, never a source of truth.

---

8. Latency, throughput, and where async fits

> **Interviewer:** Nobody's waiting on a response here — so where does latency matter, and what moves off the critical path?

Candidate: There's no end-user SLA, which simplifies things versus the other systems in this guide. A **per-fetch timeout** (e.g. 10s) still matters — a half-dead host must never hang a worker and starve its slot in that domain's rate budget. Otherwise, everything past "bytes received" is async, since nothing waits on it:

```
worker fetches page (bounded by timeout) -> store raw HTML -> object storage (durable)
      -> parse HTML, extract links -> frontier (CPU-bound, embarrassingly parallel)
      -> emit "page crawled" event -> Kafka -> [ search indexer reads async ]
                                            -> [ link-graph / analytics jobs ]
```

Throughput, not latency, is the real metric — pages/sec sustained, bounded by bandwidth and the sum of every domain's politeness budget. The one latency lever that matters for design: cutting per-fetch overhead (DNS, TLS handshake, robots lookup) raises effective pages/sec per worker — exactly why the caches in section 7 exist.

---

9. Consistency model

> **Interviewer:** What's strongly consistent here, and what's allowed to be sloppy?

Candidate: Nothing here needs cross-machine strong consistency the way a payment or a scarce-inventory counter does. The closest invariant is "don't lose a URL forever" and "avoid — but tolerate — double-fetching."

- **Leases** give an approximate "not two workers at once" guarantee: pulling a URL grants a time-limited lease (`IN_PROGRESS` , e.g. 5 min); an expired lease returns it to the pool. Under a lease-expiry race two workers could rarely both fetch the same URL — harmless, since we **overwrite** content by `hash(url)` rather than append (idempotent by design).
- **The Bloom filter is explicitly eventual/approximate** — false positives are accepted.
- **Crawl state (status, last_crawled)** tolerates a few seconds of replica lag — no user is watching in real time.
- **The one real durability guarantee:** once content lands in object storage and its `urls` row flips to `DONE` , that write should survive — we don't want to silently lose a page we spent real bandwidth fetching.

---

10. Failure modes & graceful degradation

Failure	What happens	Mitigation
Crawler trap (infinite calendar/session-ID links)	One site could eat the entire crawl budget	Max depth from seed + max URLs/domain/pass — a hard budget per host
Slow/half-dead host	Fetch hangs, wastes a worker slot	Hard per-fetch timeout; mark `FAILED` , retry with backoff
Worker crashes mid-fetch	URL stuck `IN_PROGRESS`	Lease expires automatically -> returns to frontier

Failure	What happens	Mitigation
Frontier shard down	Domains on that shard stall briefly	Frontier replicated (e.g. 3x); a replica takes over
Bloom filter false positive	Skip a page not actually crawled	Accepted by design; self-corrects next pass
One very popular domain	Can't crawl it faster with more workers	Politeness caps it — patience, not machines
Region-wide outage	Nearby sites delayed	Other regions keep working; delayed, not lost

Overriding principle, unlike a giveaway or flash sale: this is a long-running background job, not a user-facing request — almost every failure resolves to "retry later, keep moving," never "fail loudly and stop."

---

11. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
URL frontier	**Kafka (or similar), partitioned by domain**	Durable, ordered-per-partition, horizontally scalable; domain partitioning gives a free place to enforce politeness. *Not an in-memory queue* — dies on crash, doesn't scale past one box, no per-domain isolation.
Dedup / "already seen"	**Bloom filter**, sharded like the frontier	O(1) memory-only checks at billions-of-entries scale, tunable false-positive rate. *Not a* `HashSet` *or per-check DB* `SELECT` — a hash set that size doesn't fit one machine, and 500B DB round trips for an approximate question melts a store meant for real crawl-state traffic.
URL metadata / crawl state	**Distributed KV / wide-column store** (DynamoDB/Cassandra/Bigtable-style)	Pure single-key lookup/update at huge volume, no joins. *Not SQL* — no relational need, and one SQL instance can't hold billions of rows this cheaply.
Page content	**Object storage** (S3-like), keyed by `hash(url)`	Big blobs, write-once, read-by-key, near-infinite and cheap, replicated by default. *Not a database* — storing hundreds of TB-PB of blobs in a DB wastes cost for no query benefit.
Per-domain rate limiting	**Shared token bucket per domain**	Naturally shared once the frontier is domain-partitioned. *Not per-worker limits* — N workers each self-limiting to rate R still sums to N×R against the site.
Robots.txt / DNS	**Local/shared cache, TTL-based**	Both change rarely; per-page re-fetch would double request volume to every site on earth.
Crash recovery	**Time-limited leases**	Automatic — an expired lease just returns the URL; matches "retry later, keep moving."

Concern	Choice	Why this, and why not the alternative
Near-duplicate detection	**Content hash / SimHash on the body**	Catches mirrors/tracking-param duplicates a URL-keyed filter structurally can't.
Control-plane calls	**Internal gRPC** (worker <-> robots-cache service)	Fast, typed, service-to-service — but minor; there is **no public-facing API**, since nobody outside the pipeline calls it.

Say-it-out-loud justification: "*The frontier is a durable, domain-partitioned queue so politeness is always enforced in exactly one place per domain, no matter how many of the ~500 stateless workers exist; a Bloom filter gives a cheap, approximate 'have I seen this?' check that would otherwise melt a database at billions of lookups; metadata lives in a sharded KV store because it's pure single-key access, while bodies live in cheap object storage because they're big blobs read by key; and because nobody is waiting on a response, the whole system optimizes for steady, polite, fault-tolerant throughput over raw speed.*"

---

12. Follow-up curveballs

> **Interviewer:** How do you decide crawl *priority* — what gets fetched first?

Candidate: Score on insert by `importance + staleness`: importance from inlink count (a simplified PageRank signal), staleness from time since last crawl. That score picks the priority tier (section 5). Homepages and heavily-linked pages bubble up; obscure deep pages sink and might wait weeks.

> **Interviewer:** What about JS-heavy sites where content only appears after client-side rendering — a plain GET gets you an empty shell?

Candidate: Don't make every worker pay for that. Route a **small, separate pool of headless-browser workers** at sites flagged as JS-heavy (an allowlist, or detecting suspiciously content-free HTML), isolated so it can't slow the fast plain-HTTP pool doing 99% of the volume.

> **Interviewer:** How do you stop this from running forever without ever "finishing"?

Candidate: It's not meant to finish — it's a continuous pipeline. Important, fast-changing pages get re-crawled every few hours; obscure static pages every few weeks, driven by the same importance/staleness score. "Done" isn't a real state here.

> **Interviewer:** Two workers discover the same brand-new URL at nearly the same instant — do we double-fetch it?

Candidate: Possibly once, briefly — the Bloom filter's insert-if-not-seen is a fast approximate check, not a lock, so there's a small race window. Harmless: since we overwrite content by `hash(url)` rather than

append, double-fetching wastes a little bandwidth but never causes a correctness bug — the same idempotency argument as the lease-expiry race in section 9.

---

13. Deep-dive: "Why not just a DB table or an in-memory hash set for 'already seen'?"

> **Interviewer:** Section 6 waved at this, but let's slow down. I don't buy that a Bloom filter is *necessary*. Why can't "have I seen this URL?" just be a row in Postgres, or even simpler — a `HashSet<String>` in each worker's memory? Convince me.

Candidate: Both are the obvious first instincts, and both are exactly right at small scale — the reason to reject them here is purely the *number* of URLs, not the idea itself. Let me size each one against the actual workload: ~20 billion distinct URLs live in the seen-set at steady state, with up to 25,000 new-URL checks per second.

Why an in-memory `HashSet<String>` doesn't survive this

```
Average URL length ~= 80 bytes. A Java/Go HashSet doesn't store just the
bytes -- each entry carries object overhead, a hash bucket pointer, string
headers, etc. Realistic all-in cost: ~150-200 bytes/entry.

  20,000,000,000 URLs * ~180 bytes/entry  ~=  3.6 TB

-> Doesn't fit in one machine's RAM (even a big box tops out ~1-2TB).
-> "Just shard it across workers" means every worker needs a consistent
   view of a 3.6TB structure that's also growing continuously -- you've
   reinvented a distributed cache, except you built it yourself instead
   of using one that already exists.
-> And it's gone the instant the process restarts -- no durability at all.
```

Why a DB row-per-URL lookup doesn't survive this either

```
SELECT 1 FROM urls WHERE url_hash = ? LIMIT 1;

At 25,000 new-URL checks/sec, sustained, that's 25,000 network round trips
per second just to answer a yes/no question -- before a single page is
even fetched. Each round trip is ~1-5ms even on a fast KV/SQL store.

  25,000/s * (index lookup + network hop) -> the "have I seen this" check
  alone competes with -- and can crowd out -- the real crawl-state writes
  (status=DONE, last_crawled) that same store needs to serve.
```

It's not that Postgres/DynamoDB *can't* answer this query correctly — it's that spending a guaranteed, exact round trip on a question that's allowed to be *approximate* is the wrong trade. We'd be paying for perfect accuracy on a check where near-perfect is free.

Why the Bloom filter fits

A Bloom filter answers the same question with a **small, fixed, tunable error rate** instead of a network round trip:

```
Sized for ~20B entries at 0.1% false-positive rate:
  ~10 bits/entry (rule of thumb for ~0.1% FP)
  20,000,000,000 * 10 bits ~= 200,000,000,000 bits ~= 25 GB

-> 25 GB, not 3.6 TB. Fits comfortably in memory, shardable by domain
   the same way the frontier already is. One in-memory bit-array check,
   no network hop, no disk I/O -- microseconds, not milliseconds.
```

The cost of that 1,400x memory reduction: **false positives**. Roughly 1 in 1,000 genuinely new URLs gets told "probably seen" and is silently dropped. There are **no false negatives** — a Bloom filter never says "not seen" for something it actually inserted, so we never mistake a *seen* page for new (which would waste a fetch, not lose one).

When is that false-positive rate actually acceptable?

This is the crux of the trade-off, and it's worth stating explicitly:

- **Acceptable here** because the downstream cost of a false positive is "skip one page this pass" — and the crawler re-visits the same domain again in ~2 weeks, at which point that page gets another chance (a *different* random 0.1% of the bit array collides next time, so it's not the same page being perpetually skipped). Nobody is blocked on this page existing right now; there's no user-facing request waiting on it.
- **Not acceptable** in domains where a false positive means real loss with no retry — e.g. "has this payment ID already been charged" (a false "yes" skips a legitimate charge, or worse, a false "no" double-charges) or "has this user already redeemed this one-time coupon." Those need exact membership (a DB unique constraint or `SET NX`, as in the burger-giveaway design), because there's no forgiving second pass.

Concern	DB table (`SELECT WHERE url=?`)	In-memory `HashSet`	Bloom filter
Memory/storage at 20B URLs	Lives on disk + DB cache; index alone is 100s of GB, but doesn't need to fit in RAM	~3.6 TB of RAM — doesn't fit one machine	~25 GB of RAM for 0.1% FP rate — trivially fits, shardable
Latency per check	~1-5ms (network + index lookup)	~microseconds (in-process) — *if* it fit	~microseconds (in-process bit checks)
Throughput at 25,000 checks/sec	Competes with real crawl-state traffic on the same store	Fine, but see memory problem above	Trivial — this is exactly what it's built for
Accuracy	Exact — no false positives or negatives	Exact — no false positives or negatives	Small tunable false-positive rate; **zero** false negatives
Durability across restarts	Durable (it's the DB)	None — lost on process restart	None on its own — rebuilt from a durable source, or periodically snapshotted

Concern	DB table (`SELECT WHERE url=?`)	In-memory `HashSet`	Bloom filter
Best fit when...	Exactness matters and volume is modest (thousands-millions of keys)	Exactness matters, single process, dataset is small	Volume is huge, checks are frequent, and a rare false positive is harmless/self-correcting

> **Rule of thumb:** *Use an exact structure (DB row, `HashSet`, `SET NX`) when a wrong answer causes unrecoverable harm — money, one-time redemptions, correctness invariants. Use a probabilistic structure like a Bloom filter when volume is enormous, checks are frequent, and the failure mode of "occasionally wrong" is cheap and self-healing — like skipping one page that gets picked up on the next crawl pass anyway.*

---

14. Deep-dive: politeness vs. crawler traps — when the frontier misbehaves

> **Interviewer:** Let's stress-test the frontier design from section 5. Walk me through, in detail, what actually goes wrong if (a) politeness isn't enforced correctly, and (b) the crawler hits a site that generates infinite URLs. These feel related — are they?

Candidate: They are — both are "the frontier keeps handing out URLs it shouldn't," just in two different directions. One floods a *real* site into a ban; the other floods the *crawler itself* into wasting its entire budget on a *fake infinity* of pages that don't matter. Let me walk through each failure concretely, because the naive version of the frontier is vulnerable to both.

Failure mode A — a naive frontier floods one host

Say the frontier is just one global priority queue, scored purely by importance/staleness, with **no per-domain gate** — exactly what section 5 argued against, but let's see the failure in motion:

```
Q_high (global, sorted by score, no domain awareness):
  [nytimes.com/a  nytimes.com/b  nytimes.com/c  nytimes.com/d  nytimes.com/e ...]
   ^ nytimes.com has millions of high-inlink pages -> they ALL score high
     -> they cluster at the TOP of the same global queue

500 workers, all pulling "the next highest-scored URL":

  worker 1  -> pops nytimes.com/a  -> fetch NOW
  worker 2  -> pops nytimes.com/b  -> fetch NOW      } all in the same
  worker 3  -> pops nytimes.com/c  -> fetch NOW      } ~10ms window --
  worker 4  -> pops nytimes.com/d  -> fetch NOW      } nytimes.com sees
  ...                                                 } hundreds of
  worker 500-> pops nytimes.com/z  -> fetch NOW      } concurrent requests

Result: nytimes.com sees a burst that looks exactly like a DDoS.
Best case: our IP range gets rate-limited or blocked. Worst case:
we degrade a real production site for real users. Either way, our own
crawl of nytimes.com is now dead for the rest of the run.
```

The bug isn't any individual worker — each one correctly grabbed "the best URL available." The bug is that **nothing in the system ever asked "how many requests has this domain seen in the last second?"** Importance and politeness were conflated into one score, so a popular domain's very popularity is what causes it to be hammered.

Failure mode B — a crawler trap loops forever

Now the opposite shape: instead of one real site draining the whole worker pool at once, one *pathological* site generates URLs faster than we can ever finish it, silently consuming budget forever:

```
seed: events.example.com/calendar?month=1

fetch page -> extract links -> page has a "next month" link:
  events.example.com/calendar?month=2
  events.example.com/calendar?month=3
  ...
  events.example.com/calendar?month=9999999

OR session-ID traps, worse because every link looks "new":
  shop.example.com/cart?sid=a1b2c3
  shop.example.com/cart?sid=a1b2c4     <- different string, same page,
  shop.example.com/cart?sid=a1b2c5        Bloom filter says "never seen"
  ...                                     EVERY time (URL is unique text)

Frontier's view: an endless stream of "definitely new, definitely
unseen" URLs, all technically valid, all passing every check we've
built so far (robots allows it, domain has request budget, Bloom
filter says new). Nothing here looks wrong to the system -- it just
looks like an unusually large site.

Left unchecked: this ONE domain's queue never drains. Workers keep
getting handed its URLs because the queue never runs dry, so it can
crowd out fair scheduling across the fleet, and the domain's page
count grows without bound in storage.
```

The insidious part: this isn't a bug in any single component — robots.txt, the token bucket, and the Bloom filter are all working exactly as designed. The trap exploits the fact that **"new URL" and "worth crawling" are being treated as the same thing**, and nothing bounds how much of the *entire crawl*'s attention one domain can consume.

Ranked fixes

1. Per-host politeness queues with a shared token bucket (fixes failure mode A). This is section 5's Layer 2, and it's the load-bearing fix for the flooding case: partition the frontier by domain so a router can never hand out two `nytimes.com` URLs faster than that domain's bucket allows, *regardless of how many workers are idle and eager*. This alone stops the DDoS-shaped failure — importance can push a URL to the front of *its own domain's* queue, but never lets it jump the domain-level rate gate.

2. `robots.txt` compliance, cached (necessary, but not sufficient). Respects the site's own stated limits (`Crawl-delay` , disallowed paths) — some sites explicitly disallow `/calendar` or `/cart` style paths precisely because they know these are traps for crawlers. Free protection when it's present, but plenty of trap-generating pages are never listed in `robots.txt` because the site owner didn't anticipate the trap either.

3. URL normalization + dedup (shrinks failure mode B, doesn't eliminate it). Strip known-junk query params (session IDs, tracking params like `utm_*`), lowercase the host, resolve `.` / `..` in paths, sort remaining query params — collapse `cart?sid=a1b2c4` and `cart?sid=a1b2c5` to the *same* canonical URL before the Bloom filter ever sees it. This kills session-ID-style traps outright. It does **not** help the calendar case — `month=1` vs `month=2` are legitimately different paths; normalization can't tell they lead nowhere useful.

4. Depth and per-domain budget limits (the general-purpose backstop). Cap **link depth from the seed** (e.g. don't follow more than N hops deep without a strong importance signal) and cap **total URLs crawled per domain per pass** (e.g. 100k). Once `events.example.com` hits its budget, stop enqueuing new URLs from it this pass, no matter how many "new" links it keeps offering. This is the fix that catches traps normalization *can't* pattern-match — depth and budget don't care *why* a domain has infinite URLs, only that it does.

5. Explicit trap detection (defense in depth, not the primary fix). Heuristics like "this domain's discovered-URL count is growing much faster than its importance/inlink signal justifies" or "the last 10,000 URLs from this domain share an identical page template with one incrementing parameter" can flag a domain for a stricter budget or human review. Useful as a signal, but heuristics have false positives/negatives — never the *only* line of defense.

Recommended approach

Combine #1 and #4 as the two hard guarantees, with #3 as a cheap force-multiplier and #2/#5 as supporting layers — because #1 and #4 are the only two that hold *even when the site actively works against us*:

```
every URL, before it's allowed into the frontier at all:

  normalize (strip session params, canonicalize) -> Bloom filter check
       |
       ▼
  under this domain's per-pass URL budget?  --no--> drop, log, maybe flag for review
       | yes
       ▼
  enqueue onto THIS DOMAIN's politeness queue (shared token bucket)
       |
       ▼
  worker pulls only when domain's bucket has a token
```

Politeness (#1) bounds how *fast* we can ever hit one domain — this protects the *outside world* from us. The per-domain budget (#4) bounds how *much total attention* one domain can ever consume — this protects the *crawl itself* from any one pathological site, trap or not. Neither one depends on correctly recognizing a trap in advance, which is exactly why they're the two to lead with: a budget cap stops an infinite-calendar site cold on day one, without us ever having to write a rule that says "calendars are bad."

> **What to say out loud:** "*These are the same bug wearing two faces — the frontier handing out URLs without asking 'have I already given out too many for this domain, in this second, or in this pass overall?' Per-domain token buckets answer the 'too fast' version and protect the site we're crawling; per-domain budget caps answer the 'too much, period' version and protect our own crawl from a trap. Normalization and robots.txt shrink the problem cheaply, but the two budget-based checks are the ones that hold even against a domain that's actively generating infinite URLs on purpose.*"

Summary — the mental model

A web crawler is **not "protect one hot resource from a write storm" like the burger giveaway or flash sale — it's "politely and losslessly walk a graph with billions of nodes, without ever hammering any single node too hard."** The frontier is a durable, **domain-partitioned** priority queue so politeness (rate limits, `robots.txt`) is enforced in one place per domain regardless of fleet size. A **Bloom filter** makes "have I seen this?" cheap where a real database lookup would fall over, and a content hash catches near-duplicates the URL check can't. Metadata lives in a sharded KV store (single-key access); bodies live in object storage (cheap blobs by key). Leases make crashed-worker recovery automatic, depth/per-domain caps stop crawler traps from eating the budget, and because nobody is ever waiting on a response, every failure resolves to "retry later, keep moving" rather than "fail loudly."